

# 智慧银行实验教程

## 实验九/十：BMAD 方法论与 CRM 全栈开发

姓名 学号 班级

2026 年 6 月 25 日

### 1 实验目的

1. 理解 BMAD (Build-Measure-Analyze-Decide) 敏捷 AI 驱动开发框架的核心理念与角色分工；
2. 掌握 BMAD 六大核心角色的职责（产品经理、业务分析师、架构师、UX 设计师、开发者、技术文档师）及 AI 模拟方式；
3. 完成从 PRD 需求文档、架构设计、UX 设计到 Epic 拆分的完整需求分析流程；
4. 体验 Sprint 迭代规划与 Story 驱动开发，理解敏捷开发的核心实践；
5. 通过商业银行 CRM 系统案例，实践前后端分离架构（React + Express）、JWT 认证、数据库设计等全栈工程技能；
6. 学习工程实践中的降级策略（数据库不可用时的内存模式替代）；
7. 完成从本地开发到 CNB 推送的版本控制全流程。

## 2 实验环境

表 1: 实验环境一览

| 类别      | 项目                   | 版本/说明         |
|---------|----------------------|---------------|
| 操作系统    | Windows 10/11 (x64)  | —             |
| AI IDE  | WorkBuddy / Qoder    | BMAD 技能集成     |
| Node.js | 托管版本                 | v22.22.2      |
| 包管理器    | npm                  | 10.9.7        |
| BMAD 框架 | npx bmad-method      | v6+           |
| 后端框架    | Express 4            | RESTful API   |
| 前端框架    | React 18 + Vite 4    | 单页应用          |
| 认证      | JWT (JSON Web Token) | 无状态认证         |
| 数据库     | 内存 Map (降级)          | 目标 PostgreSQL |
| 版本控制    | Git + CNB            | cnb.cool      |

## 3 BMAD 方法论概述

### 3.1 什么是 BMAD

BMAD (Business Manager, Architect, Developer) 是一套 **AI 驱动的敏捷开发方法论**。其核心思想是通过 AI 模拟不同的专业角色，将软件开发从「单人编程」升级为「多角色协作」的团队工作模式。

### 3.2 六大核心角色

表 2: BMAD 角色分工

| 角色             | BMAD 技能                  | 职责            |
|----------------|--------------------------|---------------|
| 产品经理 (John)    | bmad-prd                 | 需求发现、PRD 文档编写 |
| 业务分析师 (Mary)   | bmad-domain-research     | 市场调研、领域研究     |
| 架构师 (Winston)  | bmad-create-architecture | 技术选型、架构设计     |
| UX 设计师 (Sally) | bmad-ux                  | 用户体验设计        |
| 开发者 (Amelia)   | bmad-dev-story           | Story 开发、代码实现 |
| 技术文档师 (Paige)  | bmad-agent-tech-writer   | 知识管理、文档维护     |

### 3.3 BMAD 与课本理论的对应

课本第 9 章详细介绍了 BMAD 方法论在金融科技项目中的应用：

1. **Build（构建）**：根据 PRD 和架构设计，逐步构建系统模块；
2. **Measure（度量）**：通过测试用例和用户反馈收集运行数据；
3. **Analyze（分析）**：分析度量数据，识别瓶颈和改进空间；
4. **Decide（决策）**：基于分析结果决定下一步迭代方向。

## 4 实验步骤

### 4.1 步骤 1：BMAD 框架安装与环境准备

```
# BMAD 框架已集成在 WorkBuddy/Qoder IDE 中
# 如果使用其他 IDE，可通过以下命令安装：
npx bmad-method install

# 验证安装
npx bmad-help
```

可用技能列表：

- bmad-prd —产品需求文档
- bmad-create-architecture —架构设计
- bmad-ux —UX 设计
- bmad-create-epics-and-stories —Epic 拆分
- bmad-sprint-planning —Sprint 规划
- bmad-dev-story —Story 开发

### 4.2 步骤 2：创建产品需求文档（PRD）

使用 BMAD 的 PRD 技能，通过引导式问答为商业银行 CRM 系统生成完整需求文档。

项目主题：商业银行 CRM 系统

核心功能需求：

- 客户管理：注册、登录、信息管理、等级体系（普通/白银/黄金/钻石/战略伙伴）
- 贷款申请：产品选择、额度评估、审批流程
- 即申即贷：AI 预审 + 快速放款
- 还款管理：还款计划、自动扣款、逾期处理
- 增值服务：数据分析、精准营销、绩效评估

**PRD 产出（8 大章节）：**

1. 产品概述—系统定位、目标用户、核心价值
2. 功能需求—客户管理、贷款申请、审批流程等
3. 非功能需求—性能、安全、可用性
4. 用户角色—普通客户、客户经理、风控审批员、管理员
5. 客户等级体系—五级分层，差异化服务
6. 业务流程—注册 → 登录 → 申请 → 审批 → 放款
7. 技术约束—技术栈选型说明
8. 验收标准—功能完成度定义

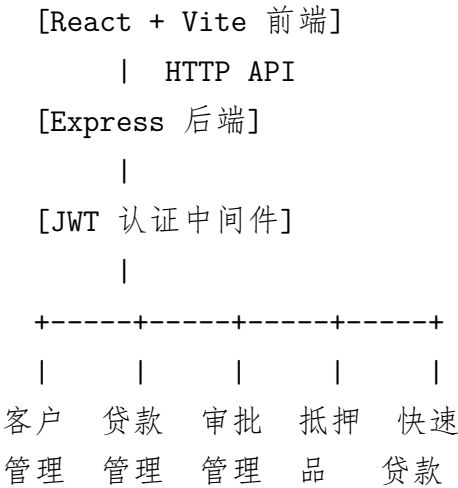
**4.3 步骤 3：技术架构设计**

基于 PRD，使用 BMAD 架构师角色设计系统的技术架构。

表 3: 技术栈决策

| 层级   | 技术         | 选型理由       |
|------|------------|------------|
| 前端框架 | React 18   | 组件化、生态成熟   |
| 构建工具 | Vite 4     | 极速 HMR、零配置 |
| 后端框架 | Express 4  | 轻量、灵活、社区大  |
| 认证方案 | JWT        | 无状态、易扩展    |
| 数据库  | PostgreSQL | 关系型、支持复杂查询 |

系统架构图：



4.4 步骤 4：UX 设计

使用 BMAD 的 UX 设计角色，生成用户旅程图和界面规范。  
设计产出：

- 用户旅程图：从注册到贷款发放的完整流程
- 页面线框图：登录/注册、客户仪表盘、贷款申请、快速贷款
- 交互规范：表单验证、加载状态、错误提示
- 视觉风格：银行级专业风格，蓝白配色方案

4.5 步骤 5：Epic 与 Story 拆分

将 PRD 中的 26 个需求拆分为 5 个 Epic：

| 表 4: Epic 拆分方案 |                 |         |
|----------------|-----------------|---------|
| Epic           | 内容              | Story 数 |
| Epic 1: 客户管理   | 注册、登录、信息管理、等级体系 | 7       |
| Epic 2: 贷款核心   | 产品管理、申请、审批、抵质押品 | 6       |
| Epic 3: 即申即贷   | AI 预审、快速放款、实时通知 | 5       |
| Epic 4: 还款管理   | 还款计划、自动扣款、逾期处理  | 4       |
| Epic 5: 增值服务   | 数据分析、营销、绩效      | 4       |
| 合计             |                 | 26      |

Story 示例（Story 1.1——用户注册）：

作为一位潜在客户，我希望能够通过手机号注册账户，以便开始使用银行的数字化服务。

验收标准：

- 1. 输入手机号和密码
- 2. 手机号格式验证
- 3. 注册成功后自动登录
- 4. 返回 JWT 令牌

4.6 步骤 6: Sprint 规划

将 26 个 Story 分配到 4 个 Sprint，制定迭代开发计划：

表 5: Sprint 规划

| Sprint   | 主题         | Stories | 预估人日 |
|----------|------------|---------|------|
| Sprint 1 | 基础服务（客户管理） | 7       | 12   |
| Sprint 2 | 贷款核心       | 7       | 14   |
| Sprint 3 | 即申即贷       | 6       | 13   |
| Sprint 4 | 增值服务       | 6       | 12   |

4.7 步骤 7: 后端实现

项目结构：

```
backend/  
  src/  
    db/  
      index.js    # 数据库连接（内存Map降级方案）  
      migrate.js  # 表结构定义（11张核心表）  
    routes/  
      customers.js # 客户管理路由  
      loans.js     # 贷款申请路由  
      approvals.js # 审批流程路由  
      partners.js  # 合作伙伴路由  
      collaterals.js # 抵质押品管理  
      repayments.js # 还款管理  
      tiers.js     # 客户等级体系  
      quick-loan.js # 即申即贷  
      test.js      # 测试数据初始化
```

```
middleware/  
  auth.js      # JWT认证中间件  
  index.js     # Express入口  
package.json
```

数据库设计（11 张核心表）：

| 表 6: 数据库表结构       |             |
|-------------------|-------------|
| 表名                | 用途          |
| customers         | 客户基本信息      |
| loan_applications | 贷款申请记录      |
| collaterals       | 抵质押品信息      |
| repayments        | 还款记录        |
| customer_tiers    | 客户等级（5 级分层） |
| loan_products     | 贷款产品定义      |
| approvals         | 审批记录        |
| partners          | 合作伙伴信息      |
| notifications     | 通知消息        |
| credit_scores     | 信用评分        |
| audit_logs        | 审计日志        |

4.8 步骤 8：前端实现

核心页面：

- 登录/注册：手机号 + 密码，JWT 认证
- 客户仪表盘：个人信息、贷款状态、等级展示
- 贷款申请：产品选择、金额输入、提交审批
- 快速贷款：即申即贷（AI 预审 + 快速放款）

Vite 代理配置：

```
// vite.config.js  
export default defineConfig({  
  plugins: [react()],  
  server: {  
    port: 5173,  
    proxy: {
```

```
    '/api': {
      target: 'http://localhost:3001',
      changeOrigin: true,
    },
  },
},
});
```

## 4.9 步骤 9：数据库降级方案

当 PostgreSQL 不可用时（ECONNREFUSED），系统自动降级为内存 Map 模式：

```
// 内存Map模拟数据库
const mockData = {
  customers: new Map(),
  loans: new Map(),
  approvals: new Map(),
  // ...其他表
};

const pool = {
  query: async (sql, params) => {
    // 模拟SQL查询逻辑
    return { rows: [] };
  },
};
```

**降级策略的教学意义：**工程实践中，理想方案（PostgreSQL）可能因环境限制无法实施。此时需要快速找到可用的替代方案（内存 Map），保证核心功能的可用性。生产环境中应使用正式的数据库和加密方案（如 bcrypt），而非 Base64 替代。

## 4.10 步骤 10：功能测试验证

后端 API 测试：

```
# 初始化测试数据（4个测试账号）
curl http://localhost:3001/api/test/init

# 测试登录
curl -X POST http://localhost:3001/api/customers/login \
  -H "Content-Type: application/json" \
```

```
-d '{"phone":"13800138001","password":"password123"}'
```

# 获取客户列表（需JWT）

```
curl http://localhost:3001/api/customers \  
-H "Authorization: Bearer <TOKEN>"
```

测试账号：

表 7: 测试账号一览

| 手机号         | 密码          | 客户等级 | 信用分 |
|-------------|-------------|------|-----|
| 13800138001 | password123 | 钻石   | 820 |
| 13800138002 | password123 | 黄金   | 750 |
| 13800138003 | password123 | 白银   | 680 |
| 13800138004 | password123 | 普通   | 600 |

#### 4.11 步骤 11: Git 版本控制与 CNB 推送

```
# 初始化 Git 仓库  
git init  
  
# 创建 .gitignore  
# （排除 node_modules/, dist/, .env, *.log 等）  
  
# 添加并提交  
git add -A  
git commit -m "feat: 添加完整项目代码（前后端+BMAD文档）"  
  
# 配置远程仓库（Token认证）  
git remote add origin https://cnb:<TOKEN>@cnb.cool/sichuanagr202301729/  
zhihuiyinhang.git  
  
# 推送  
git push -u origin master
```

#### 4.12 步骤 12: Vercel 部署（前端）

```
# 安装 Vercel CLI  
npm install -g vercel
```

```
# 登录（浏览器授权）
vercel login

# 部署
cd frontend
vercel --prod --yes
```

部署后前端可通过 `https://smart-crm-xxx.vercel.app` 公网访问。

## 5 实验结果

### 5.1 BMAD 文档产出

表 8: BMAD 方法论文档产出汇总

| 文档类型         | 产出文件             | 内容概述                   |
|--------------|------------------|------------------------|
| 产品需求文档       | prd.md           | 8 大章节，含 5 级客户等级体系      |
| 架构设计         | architecture.md  | 前后端分离 + JWT 认证方案       |
| UX 设计        | ux/DESIGN.md     | 用户旅程图 + 交互规范           |
| Epic & Story | epics-stories.md | 5 Epic / 26 Story 完整拆解 |
| Sprint 计划    | sprint-plan.md   | 4 Sprint 迭代路线图         |

### 5.2 代码产出

表 9: CRM 系统代码产出

| 模块     | 文件/功能                                                                         |
|--------|-------------------------------------------------------------------------------|
| 后端路由   | 9 个路由文件（customers / loans / approvals / partners / collaterals / repayments / |
| 后端基础设施 | Express 入口 + 数据库适配 + 迁移脚本                                                     |
| 前端页面   | App.jsx（集成登录/注册/仪表盘/贷款申请/快速贷款）                                                |
| 数据库设计  | 11 张核心表完整 ER 设计                                                               |

## 6 问题与解决

### 6.1 问题 1: PostgreSQL 不可用导致后端无法启动

现象：启动后端时抛出 `ECONNREFUSED 127.0.0.1:5432`，数据库连接失败。

**原因分析：**开发环境中未安装 PostgreSQL 服务，或服务未启动。

**解决：**实施数据库降级方案——将数据库层替换为内存 Map 模拟实现。同时将 bcrypt 密码哈希替换为 Base64 替代方案。生产环境需恢复正式方案。

**教学启示：**降级策略是工程实践的重要思维模型。在金融业务系统中，类似思路可用于设计灾备切换、限流降级等容错机制。

## 6.2 问题 2：bcrypt 模块安装失败（EPERM）

**现象：**`npm install bcrypt` 报权限错误，无法编译原生模块。

**解决：**使用 Base64 编码替代 bcrypt 进行密码哈希处理。注意这只是开发环境的临时方案，生产环境必须使用加盐哈希。

## 6.3 问题 3：CNB OAuth 登录失效

**现象：**使用 `cnb login` 登录后，`git push` 仍报认证错误。

**原因分析：**OAuth 令牌过期或浏览器会话失效。

**解决：**改用 PAT（Personal Access Token）认证方式，将令牌直接嵌入 Git 远程 URL 中。

# 7 实验心得

**BMAD 方法论的理解：**

1. BMAD 的角色分离理念（PM / 架构师 / UX / 开发者各司其职）让软件开发从「个人英雄主义」走向「团队工程协作」；
2. PRD → 架构设计 → UX → Epic 拆分 → Sprint 规划 → 代码实现，这一完整链路确保了「先想清楚再动手」，避免了边想边改的低效模式；
3. BMAD 的 Build-Measure-Analyze-Decide 四阶段循环与金融风控中的 PDCA（Plan-Do-Check-Act）循环异曲同工——都是通过持续反馈驱动迭代优化。

**全栈开发的收获：**

1. 前后端分离架构（React + Express）的实践让我理解了 RESTful API 设计、JWT 无状态认证、CORS 跨域处理等关键概念；
2. 数据库表结构设计（11 张核心表）锻炼了从业务需求到数据模型的映射能力——客户等级、贷款申请、信用评分的关联设计体现了银行业务的复杂性；
3. Vite 开发代理（proxy）的配置让前后端在开发阶段无缝衔接，避免了繁琐的 CORS 配置。

### 工程思维的形成：

1. 「降级策略」是本实验最能启发工程思维的部分——当理想方案不可行时，需要快速找到替代方案保证核心功能，这在金融系统的灾备设计中至关重要；
2. 版本控制（Git + CNB）不仅是代码备份工具，更是协作的基石——每一次 commit 都是项目演进的历史节点；
3. 从贪吃蛇的单文件 HTML（实验二）到 CRM 系统的多模块工程（本实验），我经历了从「写代码」到「做工程」的思维转变。

### 不足与改进：

1. 目前仅完成了 BMAD 文档生成和代码框架搭建，实际的代码实现和 CI/CD 流程需在完整课程中继续推进；
2. Vercel 部署仅覆盖了前端部分，后端尚未部署到云端（可考虑 Railway、Render 或 CloudStudio）；
3. 数据库仅使用了内存 Map 降级方案，正式项目中应迁移到 PostgreSQL 并引入 ORM（如 Prisma）；
4. 测试体系（单元测试、集成测试）尚未建立，后续需补充 Jest + Supertest 测试框架。

**与贪吃蛇项目比较：**贪吃蛇游戏是 AI 协作编程的入门案例——需求简单、架构扁平、单文件交付。CRM 系统则是 AI 辅助工程的进阶级实践——需求复杂、架构分层、多模块交付。从「AI 生成代码」到「AI 辅助架构」，这正是 BMAD 方法论推动的能力跃迁。